Digital Signal Processing in VLSI Design

Homework 3
Interpolator

Name : 陳映宬
Department : Electrical Engineering
Student ID : B11107027
Instructor : 蔡佩芸

April 15 , 2026

Contents

1 Interpolator Function Verification in Over-Sampled Cases (Step 1) 2
1.1 Linear & Second-Order Polynomial Interpolator Results 2
1.2 Error Comparison 3

2 Interpolator Function Verification in Typical Cases (Step 2) 4
2.1 Linear & Piecewise Parabolic Interpolator Results 4
2.2 Error Comparison 5

3 Bit-True Model Construction (Step 3) 6

4 Performance Degradation Analysis with BF16 Arithmetic (Step 4) 7
4.1 Error Analysis: Second-Order Farrow Interpolator with BF16 Operations . 7

5 Hardware Reduction (Step 5) 9
5.1 Original Second-Order Farrow Interpolator Block Diagram 9
5.2 Hardware Sharing Approaches 9
5.3 Revised Block Diagram 10

6 Hardware Implementation (Step 6) 11
6.1 Behavioral Simulation and Latency 11
6.2 Circuit Synthesis and Critical Path Analysis 12
6.3 Post-Synthesis Simulation 13
6.4 Error Plot: Hardware vs. MATLAB Floating-Point 14

1 Interpolator Function Verification in Over-Sampled Cases (Step 1)

The input signal is defined as

$$x_1[m] = \cos\left(2\pi\frac{m}{10} + \frac{\phi}{2}\right) + j\sin\left(2\pi\frac{m}{10} + \frac{\phi}{2}\right),$$

where $\phi = 1$. The sampling frequency is 10 times the signal frequency. Interpolation is performed in the region $10 \leq m \leq 20$ with $\mu = 0, \frac{1}{8}, \frac{2}{8}, \ldots, \frac{7}{8}$.

1.1 Linear & Second-Order Polynomial Interpolator Results

Linear interpolation formula:

$$x(m+\mu) = \mu\, x(m+1) + (1-\mu)\, x(m). \tag{1}$$

Second-order polynomial interpolation formula:

$$x(m+\mu) = C_0\, x(m) + C_{-1}\, x(m+1) + C_{-2}\, x(m+2), \tag{2}$$

where

$$\begin{cases} C_0 = (1-\mu)(2-\mu)/2 \\ C_{-1} = \mu(2-\mu) \\ C_{-2} = -\mu(1-\mu)/2 \end{cases}$$

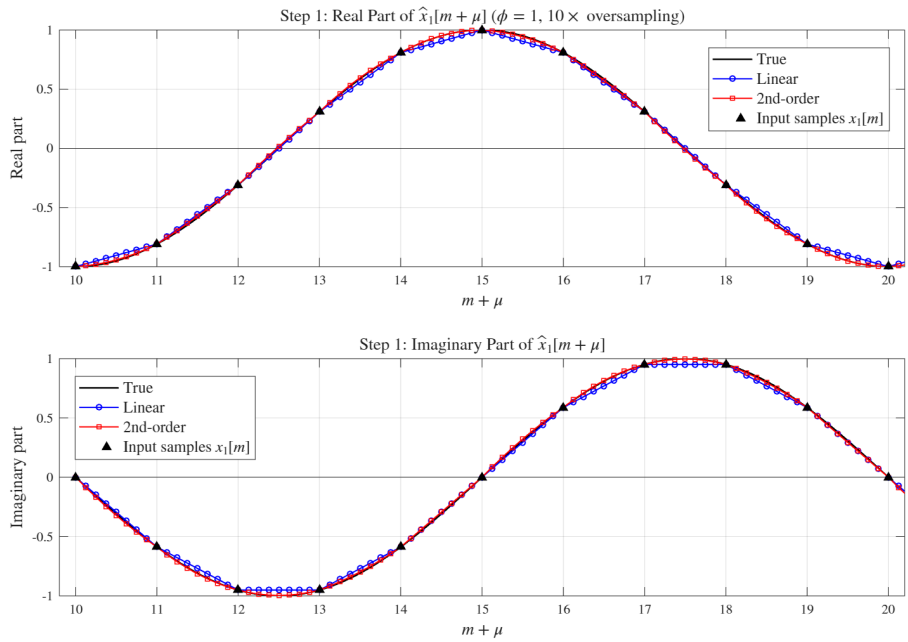

Figure 1: Real and imaginary parts: linear vs. second-order polynomial vs. true values (over-sampled case).

Analysis: Both interpolators track the true signal well at 10× oversampling. The second-order polynomial shows visibly smaller deviation at signal peaks and troughs.

1.2 Error Comparison

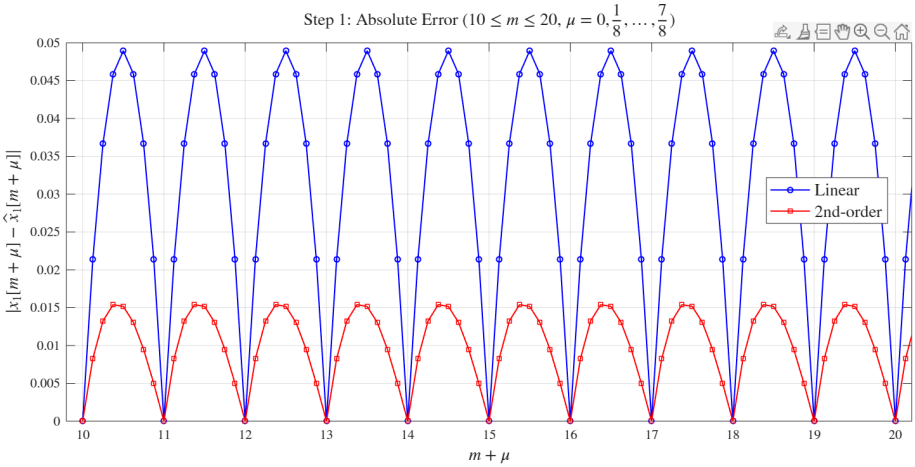

Figure 2: Absolute error comparison between linear and second-order polynomial interpolators (over-sampled case).

Table 1: Error summary for over-sampled case.

Method	**Mean |error|**	**Max |error|**
Linear interpolator	3.2103e-02	4.8943e-02
Second-order polynomial	9.9507e-03	1.5393e-02

The second-order polynomial achieves $\sim 3.2\times$ lower max error, as a quadratic fit better captures local curvature than a straight-line approximation.

2 Interpolator Function Verification in Typical Cases (Step 2)

The input signal is defined as

$$x_2[m] = 2^{-20}\left[\cos\left(2\pi\frac{m}{4} + \frac{\phi}{3}\right) + j\sin\left(2\pi\frac{m}{4} + \frac{\phi}{3}\right)\right],$$

where $\phi = 1$ (same as Step 1). The sampling frequency is 4 times the signal frequency. Interpolation is performed in the region $4 \le m \le 8$ with $\mu = 0, \frac{1}{8}, \frac{2}{8}, \ldots, \frac{7}{8}$.

2.1 Linear & Piecewise Parabolic Interpolator Results

Linear interpolation formula: Same as Eq. (1).

Piecewise parabolic interpolation formula ($\alpha = 0.5$):

$$x(m+\mu) = C_1\, x(m-1) + C_0\, x(m) + C_{-1}\, x(m+1) + C_{-2}\, x(m+2), \tag{3}$$

where

$$\begin{cases} C_1 = -\alpha\mu + \alpha\mu^2 \\ C_0 = 1 + (\alpha - 1)\mu - \alpha\mu^2 \\ C_{-1} = (\alpha + 1)\mu - \alpha\mu^2 \\ C_{-2} = -\alpha\mu + \alpha\mu^2 \end{cases}$$

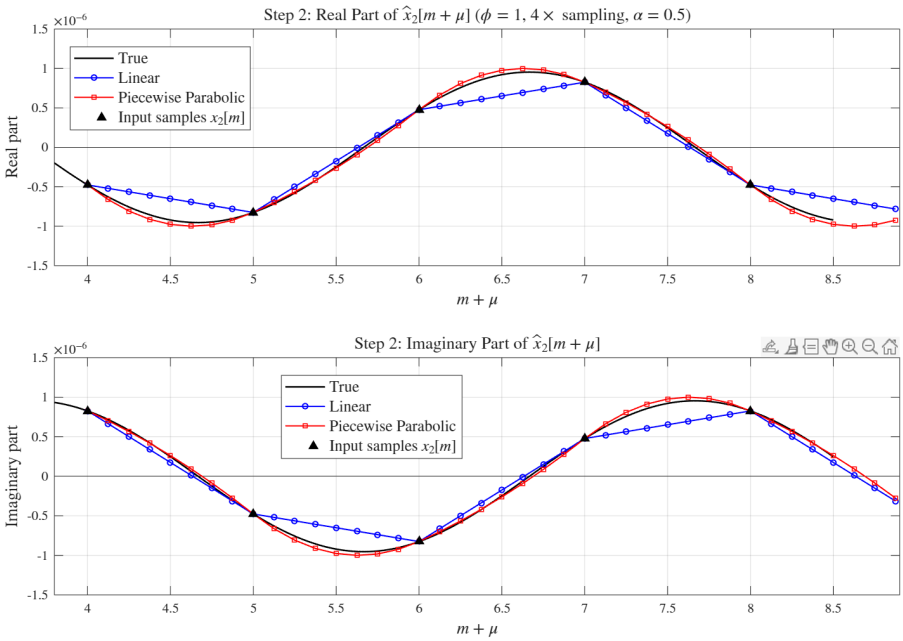

Figure 3: Real and imaginary parts: linear vs. piecewise parabolic vs. true values (typical case).

Analysis: Both interpolators reproduce the signal shape at 4× oversampling. The piecewise parabolic interpolator fits local curvature using four sample points, yielding visibly closer agreement with the true value.

2.2 Error Comparison

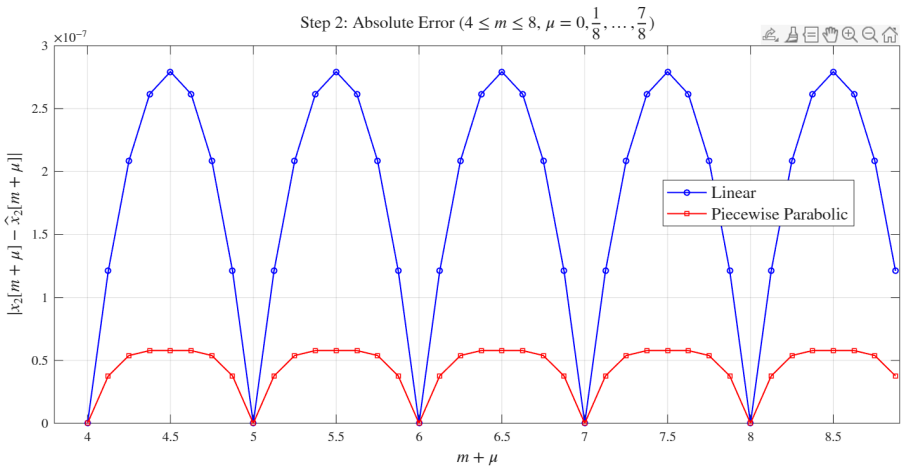

Figure 4: Absolute error comparison between linear and piecewise parabolic interpolators (typical case).

Table 2: Error summary for typical case.

Method	**Mean |error|**	**Max |error|**
Linear interpolator	1.8271e-07	2.7932e-07
Piecewise parabolic	4.4554e-08	5.7850e-08

The piecewise parabolic interpolator achieves approximately 4.1× lower error than the linear interpolator in the typical case. Although the 4× oversampling ratio is lower than Step 1, both methods achieve very small absolute errors due to the small signal amplitude (2^{-20}).

3 Bit-True Model Construction (Step 3)

Four test cases verify the BF16 bit-true model (truncation, flush-to-zero for subnormals).

	Binary (S / E / F)	**Decimal**
Op1	`1 / 1000_0011 / 0000_000`	-16
Op2	`0 / 1000_1011 / 0000_011`	$+4192$
Result	`0 / 1000_1011 / 0000_011`	$+4192$

Case (a) — Addition: The exponent gap is 8 bits, exceeding the 7-bit mantissa width, so Op1 (-16) is completely lost after alignment truncation. Result $= +4192$.

	Binary (S / E / F)	**Decimal**
Op1	`1 / 0000_0001 / 0000_011`	-1.20304×10^{-38}
Op2	`0 / 0000_0001 / 1111_010`	$+2.29589 \times 10^{-38}$
Result	`0 / 0000_0000 / 0000_000`	$+0$

Case (b) — Addition: The true sum $\approx +1.09 \times 10^{-38}$ is below the BF16 normal floor $2^{-126} \approx 1.175 \times 10^{-38}$. Flush-to-zero gives result $= \mathbf{0}$.

	Binary (S / E / F)	**Decimal**
Op1	`1 / 0000_0010 / 1100_000`	-4.11423×10^{-38}
Op2	`0 / 0111_1100 / 0000_110`	$+0.130859$
Result	`0 / 0000_0000 / 0000_000`	$+0$

Case (c) — Multiplication: The true product $\approx -5.38 \times 10^{-39}$ is also below 2^{-126}. Flush-to-zero gives result $= \mathbf{0}$.

	Binary (S / E / F)	**Decimal**
Op1	`0 / 0110_0011 / 1011_110`	$+6.46105 \times 10^{-9}$
Op2	`0 / 1001_0011 / 0101_000`	$+1.37626 \times 10^{6}$
Result	`0 / 0111_1000 / 0010_001`	$+0.0088501$

Case (d) — Multiplication: $6.461 \times 10^{-9} \times 1.376 \times 10^{6} \approx 8.89 \times 10^{-3}$. After BF16 truncation the result is $+0.0088501$, within the normal range.

4 Performance Degradation Analysis with BF16 Arithmetic (Step 4)

4.1 Error Analysis: Second-Order Farrow Interpolator with BF16 Operations

The second-order Farrow structure is used with the over-sampled signal $x_1[m]$. All arithmetic operations (multiply and add) are replaced by BF16 equivalents.

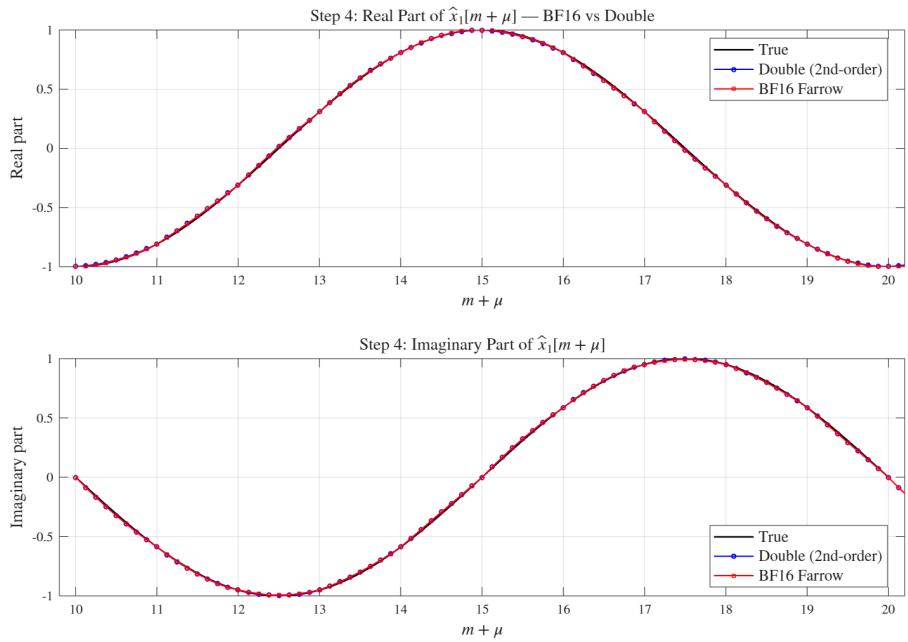

Figure 5: Real and imaginary parts: double-precision vs. BF16 Farrow interpolator vs. true values.

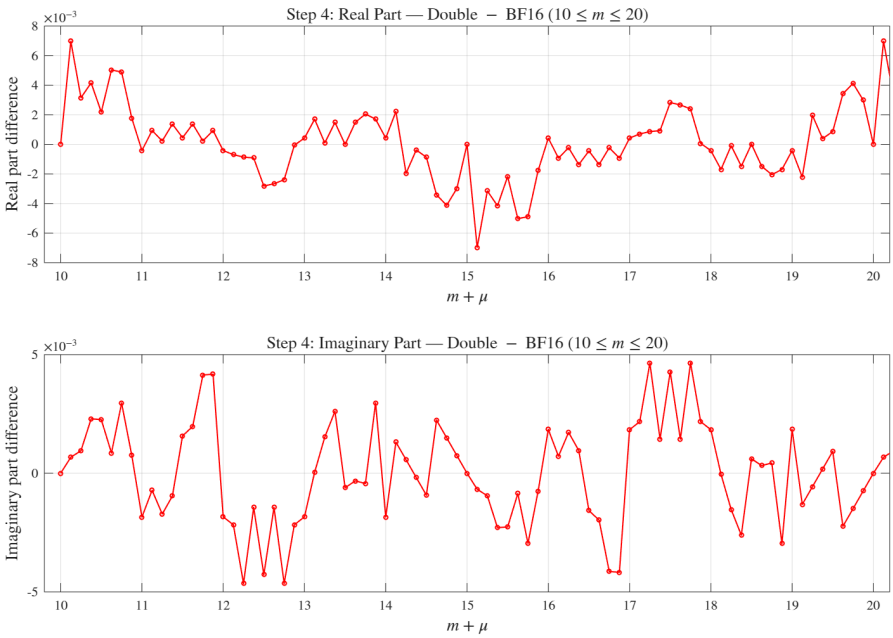

Figure 6: Difference between double-precision and BF16 Farrow outputs (real and imaginary parts).

Table 3: Difference between double-precision and BF16 Farrow outputs.

Component	**Max |diff|**	**Mean |diff|**
Real part	6.9735e-03	1.9166e-03
Imaginary part	4.6444e-03	1.7162e-03

Analysis: The maximum difference is 6.97×10^{-3}, consistent with BF16's 7-bit mantissa precision ($2^{-7} \approx 7.8 \times 10^{-3}$).

5 Hardware Reduction (Step 5)

5.1 Original Second-Order Farrow Interpolator Block Diagram

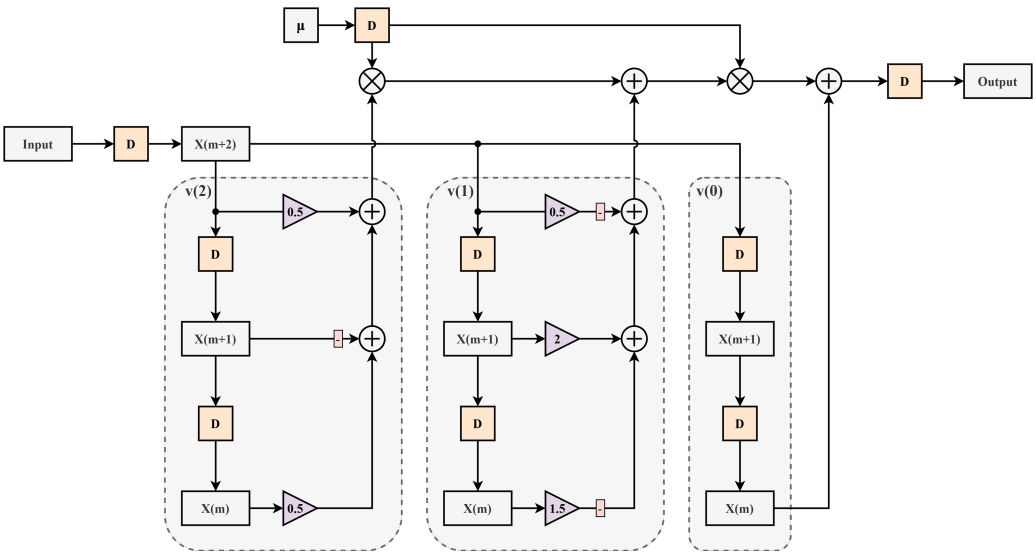

Figure 7: Original block diagram of the second-order Farrow interpolator.

The original design implements a second-order polynomial interpolator using the Farrow structure. Only three input samples, $x(m)$, $x(m+1)$, and $x(m+2)$, are required.

For each real/imaginary component: $v_2 = 0.5\,x(m) - x(m+1) + 0.5\,x(m+2)$ requires 2 multipliers and 2 adders; $v_1 = -1.5\,x(m) + 2\,x(m+1) - 0.5\,x(m+2)$ requires 3 multipliers and 2 adders; the Horner evaluation $v_0 + \mu(v_1 + \mu v_2)$ requires 2 multipliers and 2 adders.

Per component: **7 BF16 multipliers, 6 BF16 adders**.

5.2 Hardware Sharing Approaches

Three hardware reduction approaches are considered:

Approach 1 — Original Design: Each scaling coefficient ($\times 0.5$, $\times 1.5$, $\times 2$, $\times -0.5$) in the coefficient block is implemented as a dedicated BF16 multiplier, resulting in 5 multipliers per component in the coefficient block alone, plus 2 multipliers in the Horner path.

Approach 2 — Shared Delay-Line: In the original block diagram, each sub-filter path (v_2, v_1, v_0) maintains its own independent D flip-flop chain for $x(m)$, $x(m+1)$, $x(m+2)$. Since all three sub-filters read the same three consecutive samples, the duplicate delay registers are redundant and can be merged into a single shared shift register, eliminating two redundant delay paths.

Approach 3 — Exponent-Manipulation Hardware Sharing: Three observations eliminate all coefficient-block multipliers:

- $\times 0.5$ (for $0.5\,x(m)$ and $0.5\,x(m+2)$): implemented by decrementing the BF16 exponent field by 1 — a single combinational subtractor, no multiplier required.

- $\times 2$ (for $2\,x(m{+}1)$): implemented by incrementing the exponent field by 1 — similarly multiplier-free.
- $-1.5\,x(m)$: decomposed as $-(x(m) + 0.5\,x(m))$, replacing a $\times 1.5$ multiplier with one BF16 adder.
- $-0.5\,x(m{+}2)$ in v_1: reuses $0.5\,x(m{+}2)$ with a sign-bit flip (zero cost).

The coefficient block requires **zero** BF16 multipliers. Only the Horner evaluation retains 2 multipliers per component. The decomposition of $-1.5\,x(m)$ adds one extra adder per component compared to the original.

5.3 Revised Block Diagram

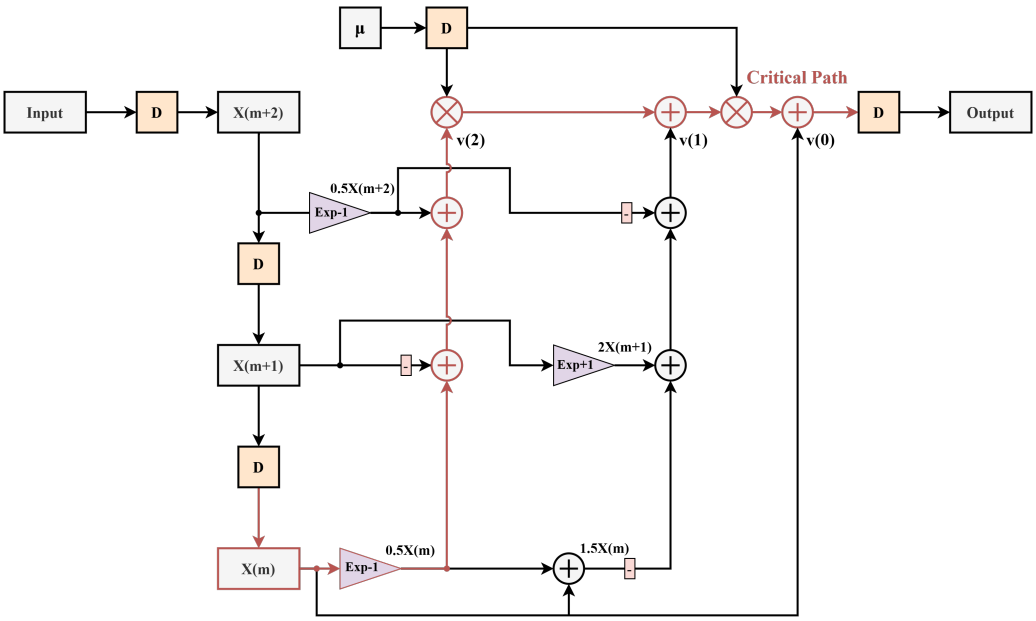

Figure 8: Revised block diagram after hardware reduction.

The revised design is mathematically equivalent to the original, confirmed by bit-exact outputs in both MATLAB and Verilog simulation. The critical path traverses the v_2 generation and Horner evaluation chain: one exponent decrement, 4 `BF16_ADD`, and 2 `BF16_MUL`.

6 Hardware Implementation (Step 6)

6.1 Behavioral Simulation and Latency

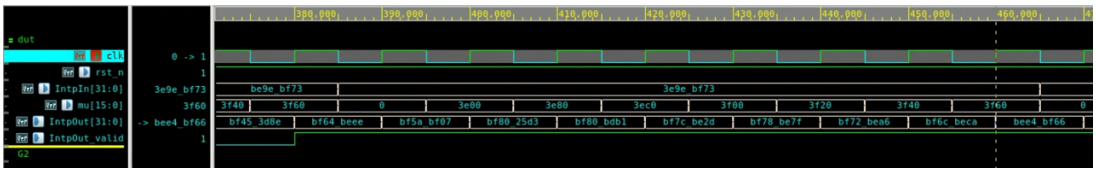

Figure 9: Behavioral simulation timing diagram — first 8 outputs.

Figure 10: Behavioral simulation timing diagram — last 8 outputs.

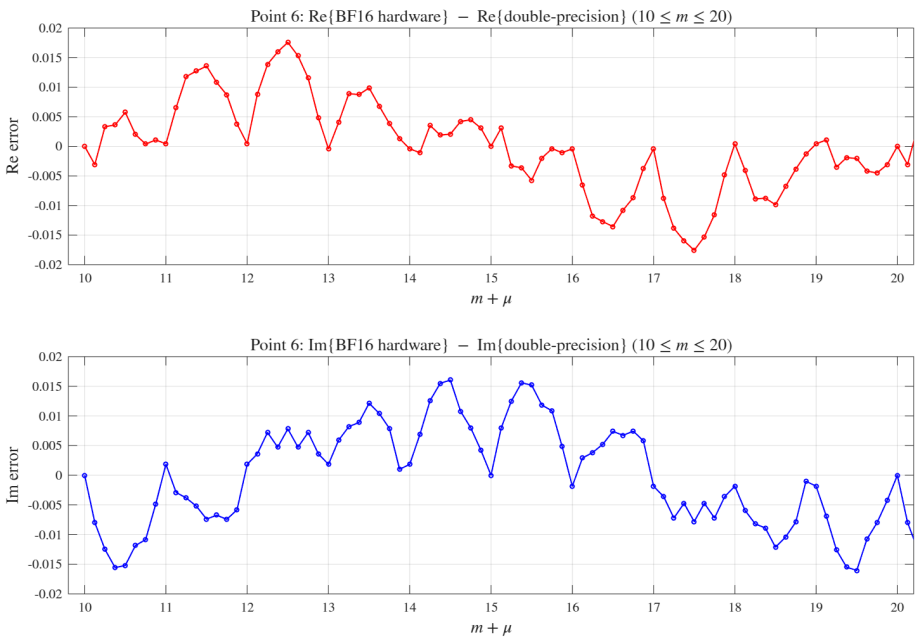

Figure 11: Point 6: BF16 hardware output (behavioral) vs. double-precision true value.

Latency: The three-tap shift register (`x0`, `x1`, `x2`) requires 3 fills driven by a mod-8 counter, giving $3 \times 8 = 24$ initialization cycles. Two pipeline register stages (input and output) add 2 cycles. Total latency from reset release to first valid output: **26 clock cycles**.

6.2 Circuit Synthesis and Critical Path Analysis

A) Theoretical Critical Path: All Farrow coefficient computation and Horner evaluation are implemented as combinational logic between the input registers (`InReg`, `mu_reg`) and the output register (`IntpOut_reg`). The longest combinational path passes through:

$$x(m) \xrightarrow{\text{EXP}-1} \underbrace{+(-x_{m+1})}_{\text{BF16_ADD}} \rightarrow \underbrace{+\,0.5\,x(m{+}2)}_{\text{BF16_ADD}} \rightarrow \underbrace{\times\,\mu}_{\text{BF16_MUL}} \rightarrow \underbrace{+\,v_1}_{\text{BF16_ADD}} \rightarrow \underbrace{\times\,\mu}_{\text{BF16_MUL}} \rightarrow \underbrace{+\,x(m)}_{\text{BF16_ADD}}$$

Theoretical critical path: **1× EXP-DEC + 2× BF16_MUL + 4× BF16_ADD** in series. The exponent decrement is a simple combinational subtractor and contributes negligible delay; the two BF16_MUL operations dominate the timing budget.

B) Post-Synthesis Critical Path: The timing constraints applied during synthesis are summarised in Table 4.

Table 4: Timing constraints applied during synthesis.

Constraint	**Value**
Clock period T_{clk}	2.0 ns (500 MHz)
Clock uncertainty	0.1 ns
Clock source latency	0.1 ns
Input max delay	0.5 ns
Output max delay	0.5 ns

Under the 2 ns clock constraint, the synthesised circuit has **no setup-time violations and no hold-time violations**; the critical path (1× EXP-DEC + 2× BF16_MUL + 4× BF16_ADD) fits within this budget. The full timing reports are shown below.

```
Startpoint: x2_reg[8] (rising edge-triggered flip-flop clocked by clk)
Endpoint: IntpOut_reg_reg[2]
          (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

Des/Clust/Port     Wire Load Model       Library
------------------------------------------------
Interpolator       ZeroWireload          N16ADFP_StdCellff0p88v125c_ccs

Point                                                    Incr       Path
-----------------------------------------------------------------------------
clock clk (rise edge)                                    0.00       0.00
clock network delay (ideal)                              0.50       0.50
x2_reg[8]/CP (DFCNQD2BWP16P90LVT)                        0.00       0.50 r
x2_reg[8]/Q (DFCNQD2BWP16P90LVT)                         0.04       0.54 f
U7038/ZN (CKNR2D4BWP16P90LVT)                            0.01       0.55 r
U7039/ZN (IND2D4BWP16P90LVT)                             0.01       0.55 f
U7040/ZN (NR3D4BWP16P90LVT)                              0.01       0.56 r
U7041/ZN (INVD4BWP16P90LVT)                              0.01       0.57 f
U7042/ZN (CKND8BWP16P90LVT)                              0.01       0.58 r
U7052/ZN (INR2D2BWP16P90LVT)                             0.01       0.58 f
U3955/ZN (CKND2BWP16P90LVT)                              0.00       0.59 r
U7547/ZN (AOI222D1BWP16P90LVT)                           0.01       0.59 f
U7550/ZN (OAI21D1BWP16P90LVT)                            0.01       0.60 r
U7556/ZN (ND3D1BWP16P90LVT)                              0.01       0.61 f
U7575/ZN (AOI211D2BWP16P90LVT)                           0.01       0.62 r
U7580/ZN (IND2D8BWP16P90LVT)                             0.01       0.64 r
```

Figure 12: Post-synthesis setup-time report — 1 (upper portion).

```
U3954/ZN (CKND2BWP16P90LVT)                                0.01       2.28 f
U9089/S (HA1D2BWP16P90LVT)                                 0.01       2.30 f
U9088/ZN (NR2D2BWP16P90LVT)                                0.01       2.31 r
U10850/Z (BUFFD1BWP20P90)                                  0.01       2.32 r
U3770/ZN (NR2D1BWP16P90LVT)                                0.01       2.33 f
U9136/ZN (AOI21D2BWP16P90LVT)                              0.01       2.33 r
U9137/Z (XOR2D2BWP16P90LVT)                                0.01       2.35 f
U9146/ZN (IND2D8BWP16P90LVT)                               0.01       2.36 f
U9158/ZN (NR3D4BWP16P90LVT)                                0.01       2.37 r
U4488/ZN (CKND2BWP16P90LVT)                                0.01       2.38 f
U11321/ZN (OAI211D1BWP20P90)                               0.01       2.39 r
IntpOut_reg_reg[2]/D (DFCNQD1BWP16P90LVT)                  0.00       2.39 r
data arrival time                                                     2.39

clock clk (rise edge)                                      2.00       2.00
clock network delay (ideal)                                0.50       2.50
clock uncertainty                                         -0.10       2.40
IntpOut_reg_reg[2]/CP (DFCNQD1BWP16P90LVT)                 0.00       2.40 r
library setup time                                        -0.01       2.39
data required time                                                    2.39
-------------------------------------------------------------------------
data required time                                                    2.39
data arrival time                                                    -2.39
-------------------------------------------------------------------------
slack (MET)                                                           0.00
```

Figure 13: Post-synthesis setup-time report — 2 (lower portion).

The hold-time report shows no violations: all hold-time slack values are non-negative under the same $T_{clk} = 2$ ns constraint.

6.3 Post-Synthesis Simulation

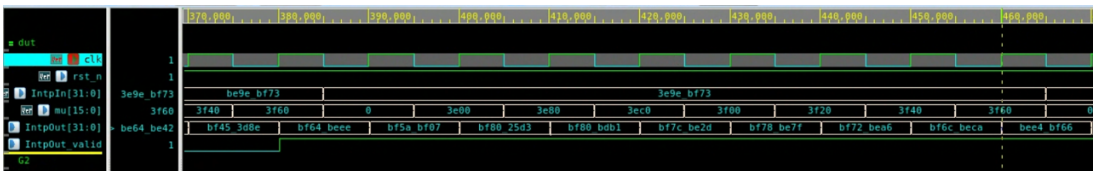

Figure 14: Post-synthesis simulation timing diagram — first 8 outputs.

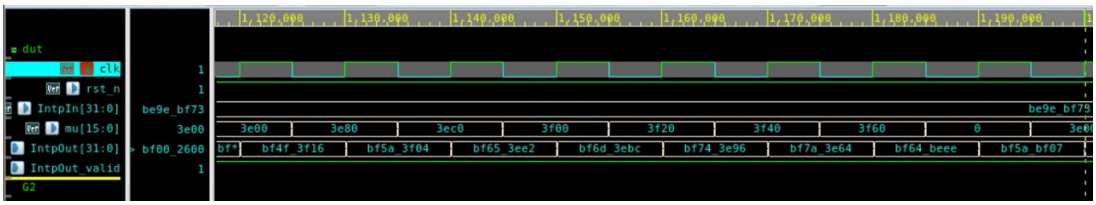

Figure 15: Post-synthesis simulation timing diagram — last 8 outputs.

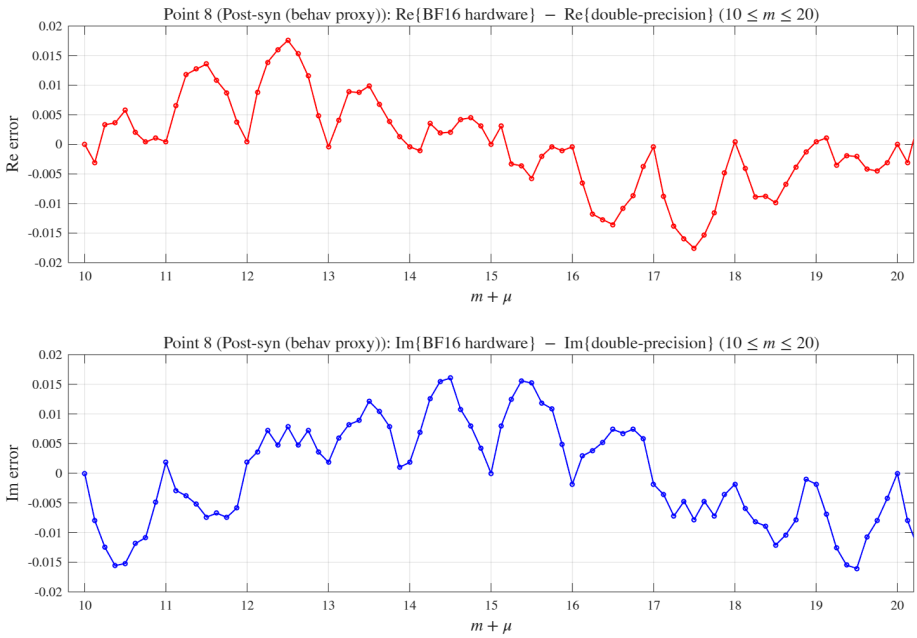

Figure 16: Point 8: BF16 hardware output (post-synthesis) vs. double-precision true value.

6.4 Error Plot: Hardware vs. MATLAB Floating-Point

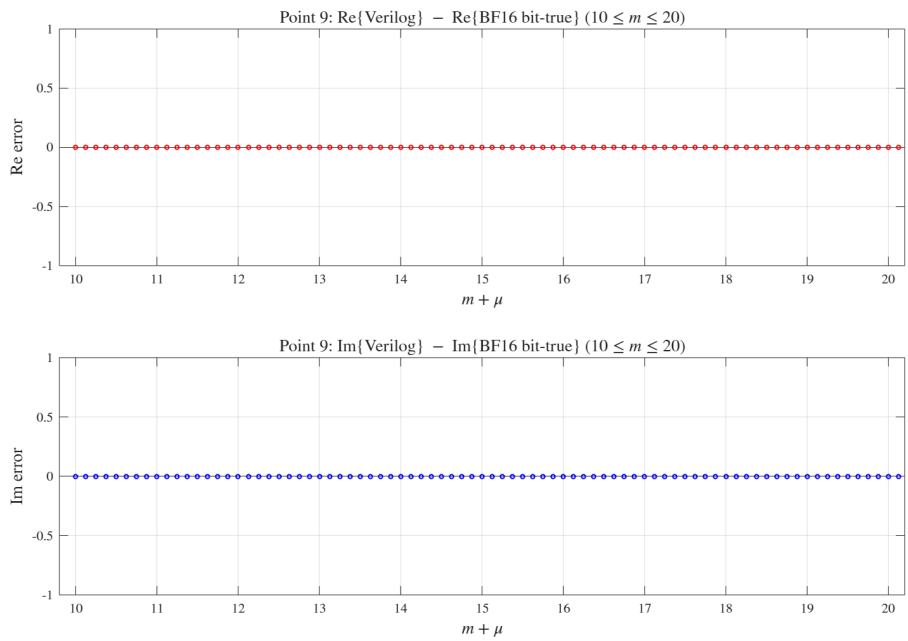

Figure 17: Point 9: error between Verilog behavioral simulation and BF16 bit-true model (all zeros confirms bit-exact match).

The Verilog behavioral simulation produces outputs that are **bit-exact** with the BF16 bit-true MATLAB model: all 176 outputs (88 real + 88 imaginary) have zero error (Point 9

verification, Section 6.1). Compared to the double-precision true value (Point 6), the BF16 hardware output has a maximum error of 1.7578×10^{-2} (real part) and 1.6048×10^{-2} (imaginary part), which is on the order of BF16 precision ($2^{-7} \approx 7.8 \times 10^{-3}$). The error is approximately 2.5× larger than the Step 4 result, because the Farrow hardware traverses a longer arithmetic chain (EXP-DEC → 4× BF16_ADD → 2× BF16_MUL), accumulating more rounding error than the direct polynomial evaluation used in Step 4.